ZOMBIE SMASHER 3D

A Course Project Report

Submitted in partial fulfillment of the course

23CS201PE407 – GAME MECHANICS, DESIGN AND DEVELOPMENT

IV Year I Semester B. Tech – CSE AIML

Submitted by

2303A52431

Under the supervision of

Dr. J. Vijaya Chandra
Associate Professor, Alumni Coordinator
Department of Computer Science & Artificial Intelligence

Academic Year: 2026–2027

SRU

Department of Computer Science and Artificial Intelligence
SR University, Warangal, Telangana, India.

Table of Contents

1. Introduction to Game Design and Development 3

2. Literature Review and Existing Game Systems 5

3. Game Concept and Project Planning 7

4. Game Mechanics Design 9

5. Game Environment and Level Design 13

6. Game Assets and Character Design 15

7. Game Architecture and System Design 17

8. Game Development Using Unity and C# 21

9. Physics, Animation and Collision Detection 24

10. User Interface and User Experience Design 26

11. Audio and Visual Effects Integration 29

12. Gameplay Implementation and Integration 31

13. Game Testing, Debugging and Optimization 33

14. Results and Performance Evaluation 36

15. Project Conclusion and Future Enhancements 39

CHAPTER 1

Introduction to Game Design and Development

1.1 Fundamentals of Digital Game Design

A digital game is a system of rules that the player explores through interaction. Game design is the work of deciding what those rules are, what the player can do, what the game does in response and how all of it feels. A widely used way to reason about this is the **Mechanics-Dynamics-Aesthetics (MDA)** framework. *Mechanics* are the rules and data written by the designer, such as "a jump lasts 0.58 seconds". *Dynamics* are the behaviour that appears when a player meets those rules, such as weaving between lanes at high speed. *Aesthetics* are the emotional responses the player has, such as tension, panic and relief. The designer controls only the mechanics directly. The player experiences the aesthetics first. In Zombie Smasher 3D, the target aesthetic is the tension of a chase, and every mechanic described in this report was chosen to support it.

1.2 Game Mechanics

Game mechanics are the concrete, testable rules of play. In an endless runner they include how the character moves, what happens on contact with each object, how score is counted, how difficulty rises and when the run ends. Good mechanics are readable (the player can predict the outcome of an action), consistent (the same action always has the same result) and tunable (expressed as numbers that a designer can adjust). For that reason all of the important values in this project, such as speeds, durations, heights and probabilities, live in a few small C# classes.

1.3 The Game Development Lifecycle

Game projects usually move through the following phases. They overlap and repeat, because playing the game always reveals new problems.

- **Concept:** the core idea, genre, audience and the experience the game should create.
- **Pre-production:** the game design document, technical choices (engine, render pipeline, platforms) and a schedule.
- **Production:** building mechanics, levels, assets, user interface, audio and effects, then integrating them into one playable build.
- **Testing:** functional, usability and performance testing, followed by balancing and bug fixing.
- **Release and post-release:** building for the target platforms, gathering feedback and adding improvements.

This project followed the same sequence, with short build-and-play loops inside production. Chapter 3 lists the milestones.

1.4 Purpose of the Project

The purpose of the project is to design and build a complete, playable 3D game that applies the concepts of the course *Game Mechanics, Design and Development*: rule design, level and environment design, asset integration, physics and collision, UI/UX, audio-visual feedback, integration and testing. An endless runner was chosen because its rules are small enough to finish within a semester, while still exercising every subsystem of a real game engine.

1.5 Objectives

- O1: Implement a third-person, three-lane runner with lane switching, jumping, sliding, stumbling and death.
- O2: Create four visually distinct themed environments that share the same core rules.
- O3: Implement zombies with different behaviours, a pursuing horde, coins and five power-ups.
- O4: Design a difficulty curve that keeps runs fair while steadily raising the challenge.
- O5: Build a complete UI flow: main menu, theme selection, HUD, pause and game-over screens, with saved best scores.
- O6: Add music, ambience, sound effects, particles and post-processing that respond to gameplay events.
- O7: Keep the game efficient through object pooling, chunk streaming, mesh merging and quality tiers.
- O8: Make the project easy to set up through an automated Unity Editor setup wizard.

1.6 Significance, Scope and Expected Outcomes

The project shows that a small team can produce a complete 3D game with many systems using only code and the engine, without buying or downloading assets. The procedural approach also keeps the project free of licensing problems, which matters for an academic submission.

Scope. Single-player and offline. It targets Windows and Android, has four themes, three playable survivors, five zombie types and five power-ups. Multiplayer, online leaderboards and monetisation are out of scope. **Expected outcomes:** a playable Unity project, a setup wizard, Windows and Android build settings, and this report.

1.7 Practical Applications

- A teaching example of how game rules map onto code: state machines, pooling, procedural generation and event-driven UI.
- A template for casual mobile games, which are the market where endless runners are most popular.
- A base for experiments in difficulty tuning and player behaviour, since all parameters are centralised and easy to log.

CHAPTER 2

Literature Review and Existing Game Systems

2.1 The Endless Runner Genre

Endless runners are games in which the character moves forward automatically and the player only steers, jumps or ducks. The run lasts until the player fails, and the goal is to beat a previous score. The genre became popular on smartphones because a run is short, the controls suit swipes and the score gives a clear reason to try again.

Table 2.1 *Representative games compared with this project*

Game	Perspective / lanes	Notable mechanics	Relevance to this project
Temple Run (Imangi Studios, 2011)	3D, third-person, path with turns	Swipe to turn, jump and slide; a pursuing monster; coins and power-ups	Established the "pursuer behind you" tension used here by the horde
Subway Surfers (SYBO / Kiloo, 2012)	3D, third-person, three lanes	Lane switching, jump and roll, coins, magnet and other power-ups	Closest structural match: three-lane layout and pickup-based power-ups
Into the Dead (PikPok, 2012)	3D, first-person	Zombie field, avoidance, limited weapons	Zombie theme and atmosphere; free movement instead of lanes
Zombie Tsunami (Mobigame, 2012)	2D side-scroller	The player is the horde, which grows by catching people	Shows the horde as a mechanic rather than only decoration

2.2 Game Engines

Unity uses C#, has a large learning ecosystem and supports one-click builds for Windows, Android and iOS. Its Universal Render Pipeline (URP) targets both PCs and phones from one project. **Unreal Engine** produces high-end visuals with C++ and Blueprints, but its projects and builds are heavier, which slows iteration for a small team targeting mobile. **Godot** is light and open source, but its 3D tooling and Android pipeline are less established. Unity 6.6 with URP was chosen because the course is taught in Unity, C# is quick to iterate in, and URP gives post-processing, fog and quality scaling on both target platforms.

2.3 Relevant Techniques and Research

- **MDA framework** (Hunicke, LeBlanc and Zubek, 2004): separates rules, behaviour and experience. It was used in Chapter 1 to set the target aesthetic.
- **Flow** (Csikszentmihalyi, 1990): engagement is highest when challenge rises together with skill. This motivates the gradual speed ramp in Chapter 4.
- **Procedural content generation** (Shaker, Togelius and Nelson, 2016): generating levels from rules and constraints rather than authoring them by hand. This project generates rows of obstacles under a "one lane must stay open" rule.

- **Object pooling** (Nystrom, *Game Programming Patterns*, 2014): reusing objects instead of creating and destroying them, to avoid garbage-collection stalls.
- **Floating origin:** moving the world back toward the origin so that floating-point precision does not degrade far from (0, 0, 0). This matters in a game that never ends.

2.4 Limitations of Existing Implementations

Commercial runners are polished, but they have limitations from a learning and design point of view, and typical student or tutorial implementations have more:

- Tutorial runners usually place obstacles at random. This can produce rows that cannot be passed, so a death feels unfair.
- Many depend on purchased asset packs, which cannot be redistributed and are hard to reproduce on another machine.
- Most use a single environment, so visual variety is low and players lose interest sooner.
- One hit usually ends the run, which punishes small mistakes as hard as large ones.
- Scene setup is manual, so moving the project to a new computer often breaks references.

2.5 How This Project Addresses Them

Table 2.2 *Limitations and the solutions used in Zombie Smasher 3D*

Limitation	Solution in this project
Unfair random rows	TrackSpawner never fills all three lanes with Block obstacles. A third obstacle is forced to be passable, and row spacing grows with speed so the free lane is always reachable.
Paid asset dependence	All meshes, textures, audio and music are generated in code. Optional free assets can be dropped in later.
Single environment	Four ThemeProfile definitions change obstacles, scenery, sky, fog, post-processing, music, ambience and footsteps.
One-hit failure	Low and high obstacles cause a stumble that brings the horde close for 6 s. Only a second stumble in that window ends the run.
Fragile manual setup	An Editor setup wizard creates the scenes, materials, prefabs and build settings, then validates them.

CHAPTER 3

Game Concept and Project Planning

3.1 Game Concept and Genre

Genre: 3D endless runner with action and survival-horror themes. **Camera:** third-person, behind and above the survivor. **One-line pitch:** "Outrun the zombie apocalypse across four ruined worlds, three lanes at a time."

3.2 Storyline

A survivor flees across a world overrun by the dead: through a fog-bound pine forest, along a desert highway lined with wrecked convoys, through rain-soaked neon streets at night, and through daylight ruins full of fire and smoke. A horde is always behind. Every 500 metres the survivor reaches a "safe zone" checkpoint, but there is no end to the road. The story is told through the environment rather than cut-scenes, which suits the short sessions of the genre.

3.3 Gameplay Objectives

- Survive as long as possible and travel as far as possible.
- Collect coins to raise the score, and power-ups to get short advantages.
- Reach safe zones to raise the score multiplier (x1.0, x1.5, x2.0 and so on).
- Beat the best score saved for each theme.

3.4 Target Audience

Casual players aged 12 and over on PC and Android who like short, replayable sessions. The game uses mild cartoon-style zombie violence with no blood or gore, which suits that audience. Four commands (left, right, jump, slide) make it easy to learn, and the rising speed keeps it challenging for experienced players.

3.5 Game Design Document (Summary)

Table 3.1 *Condensed game design document*

Item	Design decision
Platform	Windows (x86_64) and Android (IL2CPP, ARM64, landscape)
Engine	Unity 6.6 (6000.6.0f1), Universal Render Pipeline, C#
Core loop	Run, dodge, collect, survive, then retry to beat the best score
Controls	A/D or arrows to change lane; W/Up/Space to jump; S/Down to slide; Esc/P to pause; swipes on touch; gamepad supported
Worlds	Dense Zombie Forest, Post-Apocalyptic Desert, Spooky Dark City, Abandoned Apocalyptic City
Characters	Survivors: Ranger, Medic, Mechanic. Zombies: Walker, Runner, Brute, Emerger, Pursuer (horde)
Pickups	Coins; Coin Magnet, Speed Boost, Shield, Double Coins, Health Protection

Item	Design decision
Fail states	Hitting a Block or a zombie, or stumbling twice while the horde is close
Progression	Speed 11 to 25 m/s; safe zone every 500 m adds +0.5 to the multiplier; best score saved per theme
Scenes	MainMenu, ThemeSelection, ForestLevel, DesertLevel, DarkCityLevel, AbandonedCityLevel

3.6 Software and Hardware Requirements

Table 3.2 *Requirements*

Category	Requirement
Engine	Unity Hub and Unity 6.6 (6000.6.0f1) with the Universal 3D (URP) template
Packages	Universal RP, uGUI, Input System (the legacy Input Manager is also supported)
Build modules	Android Build Support (OpenJDK, SDK and NDK); Windows Build Support
Language and IDE	C#; Visual Studio or JetBrains Rider (code namespace: ZombieRunner)
Development PC	Windows 10/11 64-bit, GPU with DirectX 11 support. The recordings were made in the DX11 Editor.
Target devices	Windows PC; Android phone with ARM64 (see Chapter 13 for testing status)

3.7 Development Schedule and Milestones

The work was organised into the milestones below. Each one ended with a playable or checkable result. The status column reflects the state of the project at the time of writing.

Table 3.3 *Project milestones*

#	Milestone	Deliverable	Status
M1	Concept and design	Game concept, GDD, control scheme	Done
M2	Core runner	Lane movement, jump, slide, camera, collision	Done
M3	Content systems	Row spawner, obstacles, zombies, horde, coins, power-ups	Done
M4	Themed worlds	Four ThemeProfiles, chunk streaming, lighting, weather	Done
M5	UI and audio	Menus, HUD, pause, game over; procedural music and SFX	Done
M6	Tooling	Editor setup wizard, validation, build settings	Done
M7	Integration and play-testing	Runs in Unity 6.6 Editor in all four themes; gameplay recorded	Done
M8	Device builds and measurement	Windows/Android builds, frame-rate and memory profiling	Pending
M9	Documentation	Presentation deck and this report	Done

CHAPTER 4

Game Mechanics Design

This chapter specifies the rules of play. Every number given here is taken directly from the game source code, so the chapter also serves as a reference for tuning.

4.1 Player Movement and Running

The survivor runs forward automatically along the +Z axis. There are three lanes at x = -2.6 m, 0 m and +2.6 m. A lane change moves the survivor sideways at 15 m/s using `Mathf.MoveTowards`, so a full lane change takes about 0.17 s. Trying to move past the outer lane triggers a small camera shake as feedback instead of doing nothing. The character visually leans toward the target lane while moving.

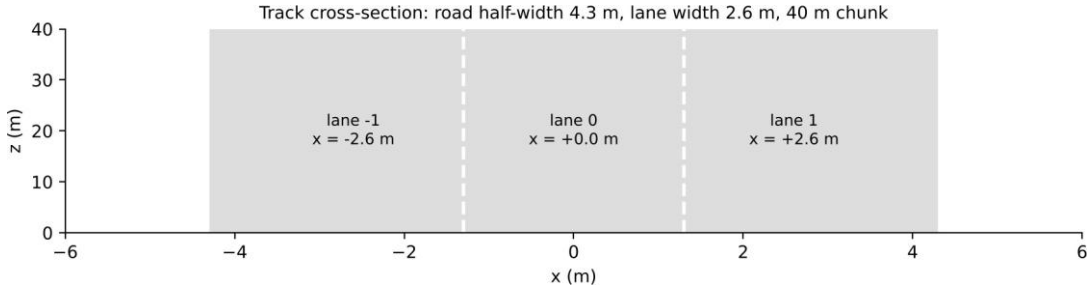


Figure 4.1 *Track layout used by all four themes (values from ZRUtil and EnvironmentGenerator).*

4.2 Jumping and Sliding

A jump sets an upward velocity of $v_0 = \sqrt{2gh}$, with gravity g = 38 m/s^2 and peak height h = 1.6 m. This gives v_0 = 11.03 m/s and an airtime of 0.58 s. Gravity is about four times Earth's, which is common in action games: the jump is snappy and readable, and its length depends mostly on run speed. A slide lasts 0.8 s and lowers the player's collision height from 1.7 m to 0.75 m. Pressing slide in mid-air triggers a **fast-fall** (downward velocity of 0.9 g) and the slide starts on landing. Pressing jump during a slide cancels the slide. Both inputs are **buffered for 0.15 s**, so a press made just before landing still counts.

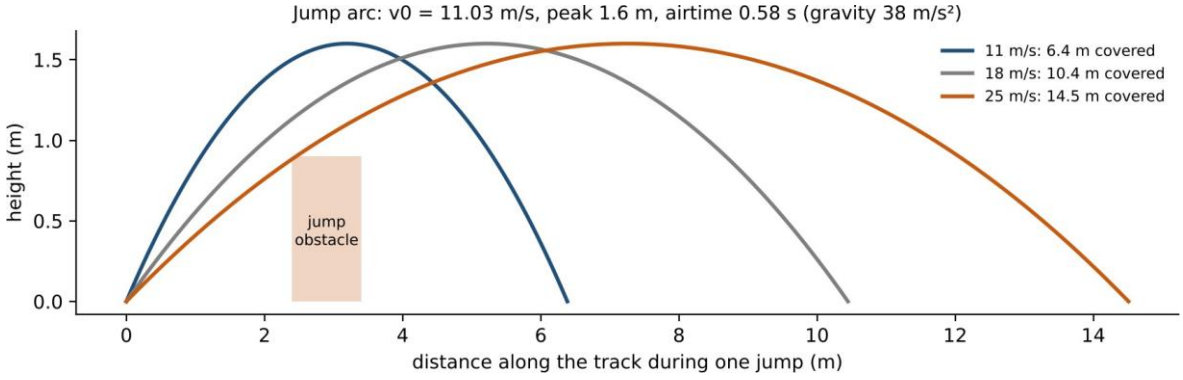


Figure 4.2 *Jump arc at three run speeds. The same jump covers 6.4 m at the start speed and 14.5 m at top speed.*

4.3 Collision Responses

Each obstacle has a *type* that decides what the player must do and what happens on contact. The result also depends on active power-ups and on whether the player hit the obstacle from the side while changing lane.

Table 4.1 *Collision rules (PlayerController.HandleHit)*

Obstacle type	How to avoid	Result of contact
Jump (low, 0-0.9 m)	Jump over	Stumble: speed x0.65 for 0.7 s, horde closes in
Pit (hole)	Jump over	Stumble
Slide (overhead, 1.1-3.1 m)	Slide under	Stumble
Block (full height)	Change lane	Death ("Crashed into ...") unless Health Protection is active
Zombie	Change lane	Death ("Grabbed by a zombie") unless Health Protection is active
Any, hit from the side while changing lane	-	Bounce back to the previous lane and stumble
Any, while Shield or Speed Boost is active	-	Obstacle is smashed away; +25 points x multiplier

4.4 Object Collection

Coins appear in lines of 5 to 9, spaced 2 m apart, in a lane that is not blocked. About 65% of rows have a coin line. If the line passes a low obstacle, the coins form an arc over it. Under an overhead obstacle they sit low, so following the coins also teaches the right move. A coin is collected within 1.15 m of the player's centre, and a power-up within 1.4 m. Power-ups appear at most once every 180 m, with a 22% chance per eligible row, and only in a completely empty lane.

Table 4.2 *Power-ups (PowerUpManager)*

Power-up	Duration	Effect
Coin Magnet	10 s	Coins within 9 m fly to the player
Speed Boost	5 s	Speed x1.45, invulnerable (smashes obstacles), field of view widens from 60 to 74 degrees, speed lines
Shield	8 s	Invulnerable; obstacles and zombies are smashed aside
Double Coins	12 s	Each coin is worth 2
Health Protection	Until used	Absorbs one otherwise fatal hit (Block or zombie)

4.5 Scoring

The ScoreManager adds points every frame and on events:

```
multiplier = 1 + 0.5 x floor(distance / 500 m)
score += distance_step x multiplier          (every frame)
score += coin_value x 10 x multiplier        (each coin)
score += 25 x multiplier                     (each smashed obstacle)
```

Because the multiplier only grows, the later and more dangerous part of a run is worth more. This rewards risk without adding new rules. The recorded runs confirm the formula. For example, the Spooky Dark City run ended with 151 m and 23 coins before the first safe zone, which gives 151 + 23 x 10 = 381, exactly the score shown on its game-over screen (Chapter 14).

4.6 Health Management: Stumble and the Horde

The game has no health bar. Instead, the player's "health" is the distance to the pursuing horde. The horde normally trails about 13 m behind. On a stumble it closes to 2.6 m for 6 s, and the screen edges turn red according to a *Danger* value from 0 to 1. A second stumble inside that window means the horde catches the player. After a stumble the player is also invulnerable to further stumbles for 1 s, so one obstacle cannot trigger two stumbles. Health Protection works as a single extra life against fatal hits.

4.7 Enemy Interactions

Table 4.3 *Zombie types (ZombieAgent and TrackSpawner)*

Type	Behaviour	When it appears
Walker	Shambles toward the player at 0.6-1.3 m/s; often in pairs, leaving one lane free	From the start
Runner	Starts 25 m beyond its row and charges down a lane at 3.5-6.5 m/s	From the start; faster at higher difficulty
Brute	Large and slow (0.6 m/s), with a wider collision box (1.4 x 2.3 m)	Difficulty level above 0.2
Emerger	Rises from the ground when the player is within 42 m, or steps in from the roadside (always in the city themes)	From the start; with an extra walker above level 0.4
Pursuer	Part of the horde behind the player (5 zombies, or 3 on Low quality)	Always present

Zombies never advance past a limit 6 m beyond the previous row. This keeps the path the player has already chosen from being blocked, so the track stays solvable. Zombies growl at random intervals using 3D sound, which warns the player before the zombie is visible.

4.8 Difficulty Progression

Forward speed follows $speed(t) = 11 + 14\ (1 - e^{-t/110})$ m/s. It rises quickly at first and then levels off toward 25 m/s. A normalised *difficulty level* (0 at 11 m/s, 1 at 25 m/s) drives the rest of the spawner: zombie-row probability rises from 16% to 30% (scaled by each theme's zombie density), the chance of two obstacles in a row rises from 25% to 60%, three-obstacle rows appear above level 0.35, and the gap between rows shrinks from about 30 m to 15 m.

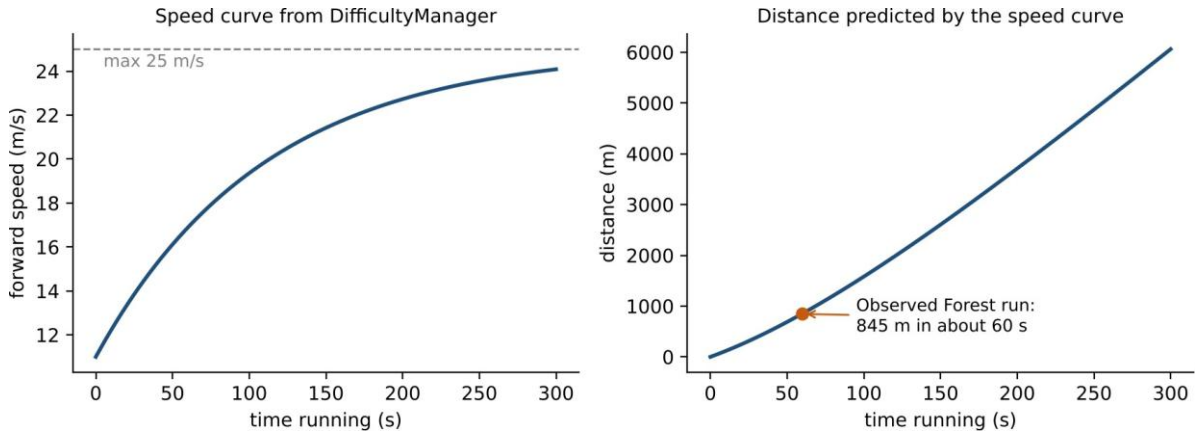


Figure 4.3 *Left: forward speed over time. Right: distance predicted by integrating that curve. The orange point is the observed Forest run.*

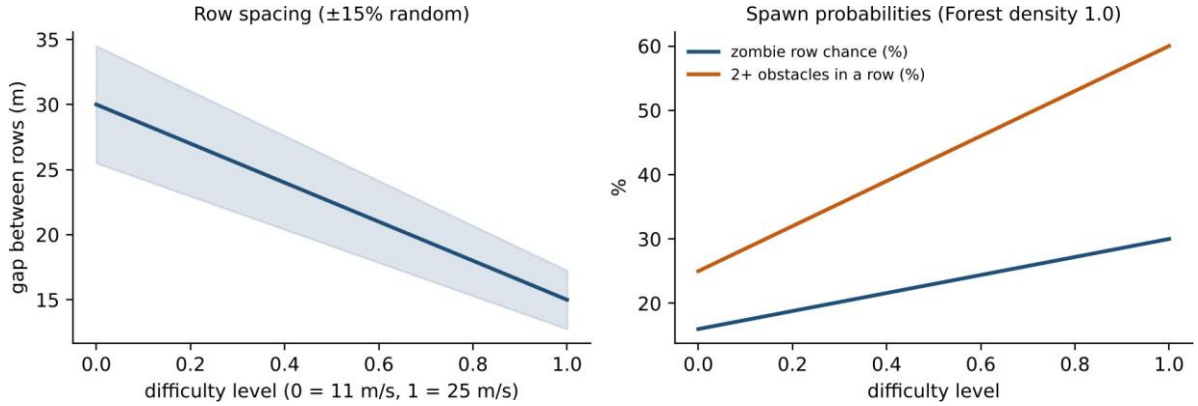


Figure 4.4 *How the difficulty level changes row spacing and spawn probabilities (computed from the TrackSpawner formulas).*

4.9 Winning and Losing Conditions

As an endless runner, the game has no final victory. The player "wins" by beating the saved best score for the theme, which the game-over screen announces with "NEW BEST SCORE!". A run is lost when the player hits a Block or a zombie without protection, or stumbles a second time while the horde is close. The game-over screen shows the cause, for example "Grabbed by a zombie" or "The horde caught you".

CHAPTER 5

Game Environment and Level Design

5.1 Procedural 3D Track Construction

The world is a straight road built from **40 m chunks**. At start-up, EnvironmentGenerator builds 8 different chunk variants per theme (6 on Low quality), each from a different random seed. It then places 6 of them in front of the player. When a chunk falls more than 25 m behind the player, it is recycled to the front, using a variant that is not currently visible. Each chunk contains the road surface with lane markings, the roadside verges and theme scenery. Its static scenery is merged into one mesh per material to reduce draw calls. A far backdrop (mountains, dunes or skylines) and the weather effects follow the player, so the horizon is always populated.

5.2 Themed Environments

Table 5.1 *Environment design of the four themes (ThemeProfile)*

Theme	Road and terrain	Atmosphere	Six obstacle types
Dense Zombie Forest	Mud trail, forest verges, pines	Fog (density 0.022), ground fog, light shafts, falling leaves	Fallen tree, wooden barricade (jump); low branch (slide); mud pit; large rock, forest wreck (block)
Post-Apocalyptic Desert	Cracked desert road, sand, dunes, cacti	Sandstorm, warm haze (0.018)	Military barricade (jump); collapsed arch (slide); sand pit; destroyed car, rock formation, abandoned truck (block)
Spooky Dark City	Wet asphalt, buildings with lit windows	Night, rain, neon signs, dense fog (0.030), reflections	Police roadblock, construction barrier (jump); fallen streetlight, scaffolding (slide); abandoned car, burning car (block)
Abandoned Apocalyptic City	Cracked asphalt, ruined buildings	Daylight haze (0.014), fires, smoke columns, ash	Rubble pile, concrete barrier (jump); fallen girder (slide); road hole; broken bus, overturned car (block)



Figure 5.1 *Dense Zombie Forest (left) and Post-Apocalyptic Desert (right) at the start of a run (frames from the gameplay recording).*

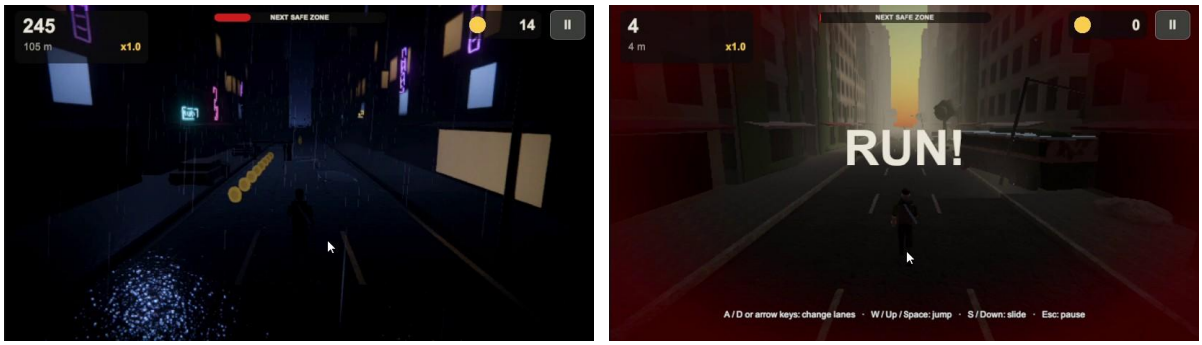


Figure 5.2 *Spooky Dark City (left) and Abandoned Apocalyptic City (right) (frames from the gameplay recording).*

5.3 Roads, Platforms and Obstacles

Because the camera always looks down the road, obstacles must be readable from far away. Each obstacle class has a distinct silhouette: low objects lie across the lane, overhead objects span it on supports, and blocks fill the full height. The gameplay trigger volume is defined by the obstacle *type*, not by the art. This means any imported model plays the same way as the procedural one, and players can learn one rule per silhouette class across all four themes.

5.4 Checkpoints, Backgrounds and Interactive Objects

Every 500 m the player reaches a **safe zone**. The HUD shows a "SAFE ZONE n REACHED - SCORE x1.5" message, a checkpoint sound plays, and the "NEXT SAFE ZONE" progress bar at the top of the screen restarts. Interactive objects are coins, the five power-ups (each with its own model, colour, glowing halo and sparkles), and obstacles that can be smashed away while the player is invulnerable. Roadside props are decoration only and are never collidable, so the player is never stopped by something that looks like scenery.

5.5 How Level Design Influences Difficulty, Navigation and Engagement

- **Fairness:** every row leaves at least one passable lane, and runners only charge down lanes that were open in the previous row.
- **Navigation:** coin lines act as guides. They show a safe lane and, by arcing or dropping, suggest whether to jump or slide.
- **Pacing:** about 8% of rows are "breather" rows with only coins, giving short moments of relief between tense sequences.
- **Engagement:** the four themes vary the look and sound while the rules stay the same, so skill carries over but the experience feels new.
- **Visibility:** fog density per theme sets how far ahead the player can see, which makes the Dark City (thickest fog) the hardest to read.

CHAPTER 6

Game Assets and Character Design

6.1 Asset Strategy

All assets in the game are original and generated in code at run time. This avoids licence restrictions and keeps the project small: the complete Unity package is about 130 KB. The game also supports optional third-party assets. Files dropped into `Assets/ZombieRunner/Imported/_Inbox` are sorted, configured and registered by the setup wizard.

Table 6.1 *Asset types and how they are produced*

Asset	Production method	Source file
24 obstacle models, coins, 5 power-up models	Procedural meshes built from boxes, cylinders, spheres and tori	PropFactory.cs, TrackItems.cs
Scenery: trees, rocks, dunes, buildings, vehicles, tents, street lights, neon signs	Procedural meshes, merged per chunk	PropFactory.cs, EnvironmentGenerator.cs
Survivors and zombies	Jointed humanoids built from capsules, animated in code	ProceduralHumanoid.cs
Textures (mud, sand, asphalt, wet asphalt, concrete, bark, foliage, rust, brick, facades) with normal maps	Generated from noise functions at 256 px (128 px on Low)	ProceduralTextures.cs, Rng.cs
Materials, skies, fog, post-processing	URP Lit, Unlit and transparent materials; volume profiles	MaterialLibrary.cs, ThemeLighting.cs
Sound effects, 5 music loops, 4 ambience loops	Synthesised at 22,050 Hz	ProceduralAudio.cs

6.2 Character Design

There are three playable survivors, the **Ranger**, **Medic** and **Mechanic**. Each has a backpack and gear and is chosen on the Character screen of the main menu. Zombies are built from the same humanoid generator with torn clothing, theme-specific colour palettes and seeded variation. Six visual variants of each zombie kind are created per theme. The Brute is scaled up so its threat is clear from its silhouette. The survivor has darker, more saturated colours than the zombies, so the player can always find their character in a crowd.

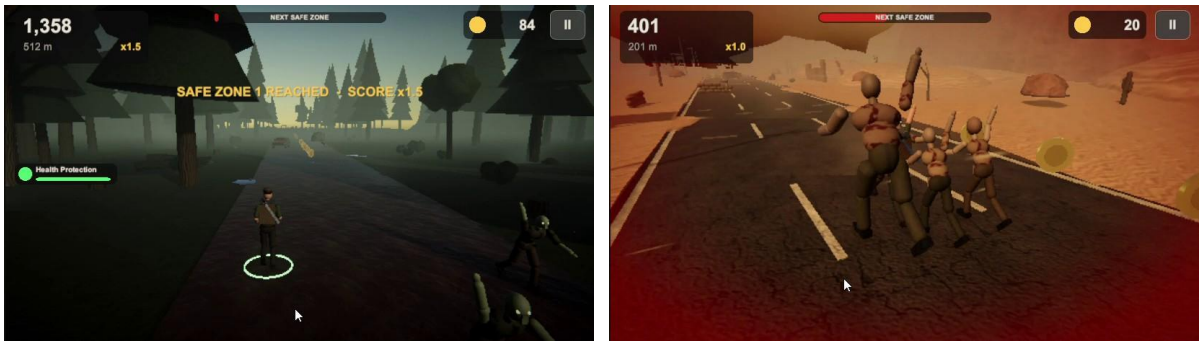


Figure 6.1 *Left: the survivor in the Forest with a zombie at the roadside. Right: Desert zombies and the horde reaching the player (recorded frames).*

6.3 Animation

Characters have eleven poses: Idle, Run, Jump, Slide, Stumble, Dead, ZombieWalk, ZombieRun, Attack, Rise and Cheer. The procedural humanoid animates its joints with sine-based cycles in code. The run cycle plays 0.9x to 1.35x faster as difficulty rises, so the character visibly works harder. If rigged models are imported (for example from Mixamo), `CharacterVisual` drives a Unity Animator with the same poses. The setup wizard builds the Animator Controller and matches clips by keywords in their file names, such as "run", "jump", "slide" and "death".

6.4 Importing, Scaling and Rendering

Imported obstacle models are scaled automatically to fit one lane (2.3 m wide) and the height of their class: 0.9 m for jump obstacles, up to 3.2 m for blocks, and lifted to 1.3 m for slide obstacles. Any colliders they include are removed, so the gameplay volume stays authoritative. All materials use URP shaders. Procedural textures include normal maps so that surfaces such as bark, asphalt and brick catch the light.

6.5 Lighting and Visual Consistency

Each theme defines a sun colour, intensity and angle, gradient ambient light (sky, equator and ground colours), fog colour and density, and a URP post-processing volume with ACES tonemapping, colour adjustments, bloom, vignette and film grain. Using one palette per theme for sky, fog, ambient light and colour filter keeps every object in a scene consistent, even though the objects are generated separately. On High quality in the Dark City, a real-time reflection probe follows the camera to give the wet road its reflections.

CHAPTER 7

Game Architecture and System Design

7.1 System Architecture

The game is organised into layers. Input is turned into abstract commands. A central **GameManager** owns the run and its state machine. Gameplay systems handle the player, spawning and enemies. Plain C# classes hold the run logic (difficulty, score and power-ups). World and presentation systems draw and play the game, and core services provide themes, pooling, saving, scene loading and quality settings. Level scenes contain only a `LevelBootstrap` component, a camera and a sun. Everything else is created by code when the scene starts.

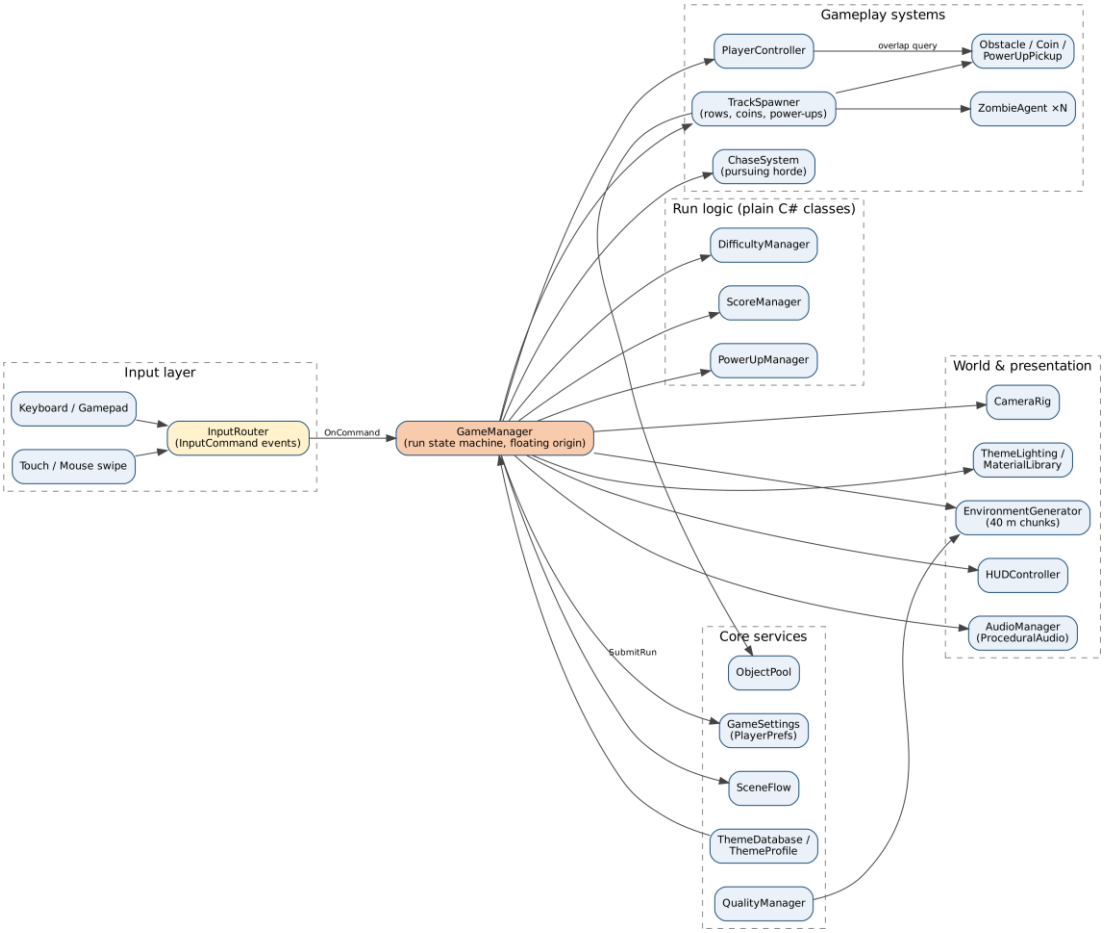


Figure 7.1 *System architecture of Zombie Smasher 3D.*

7.2 UML Class Diagram

The class diagram shows the main runtime classes. GameManager composes the subsystems for one run. TrackSpawner uses an ObjectPool and spawns obstacles, zombies and pickups. Zombies reuse the Obstacle component, so the player handles collisions with zombies and props in the same way. Pooled objects implement `IPoolable` to reset their state when reused.

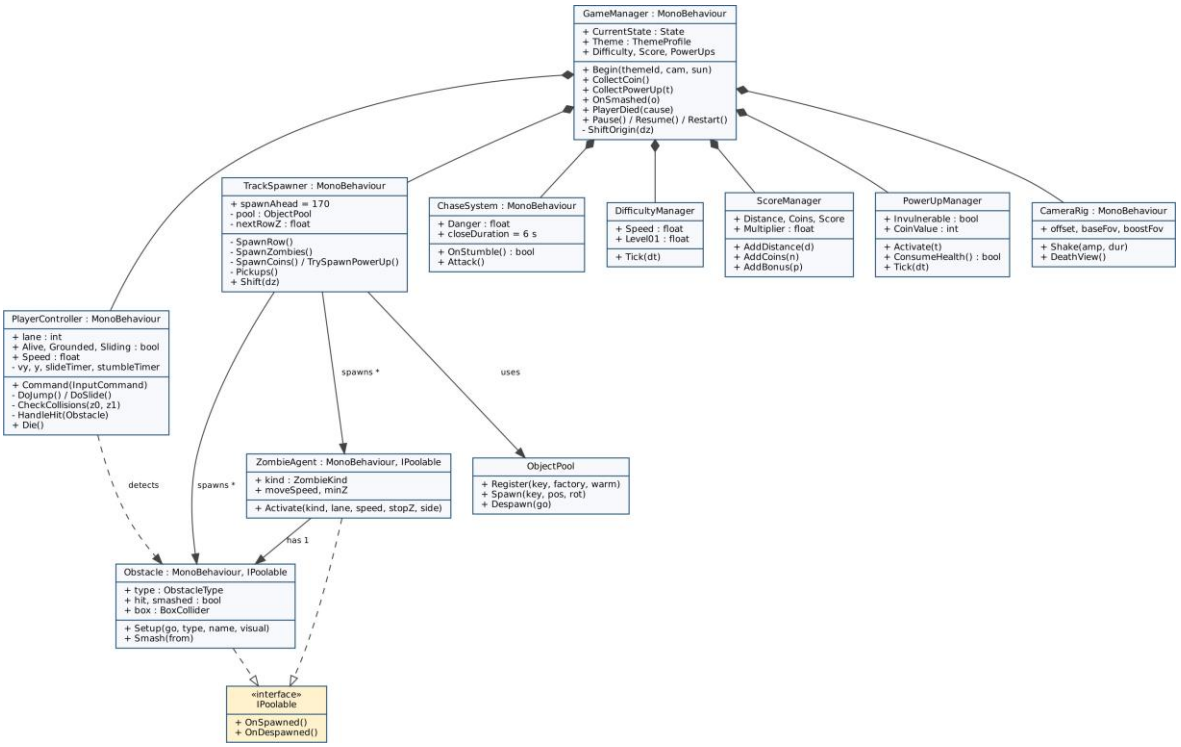


Figure 7.2 *UML class diagram of the main runtime classes.*

7.3 State Diagrams

The run is driven by a five-state machine in GameManager. Each state decides which systems are updated in that frame. Only *Running* advances difficulty, score and power-ups, and *Paused* sets `Time.timeScale = 0`.

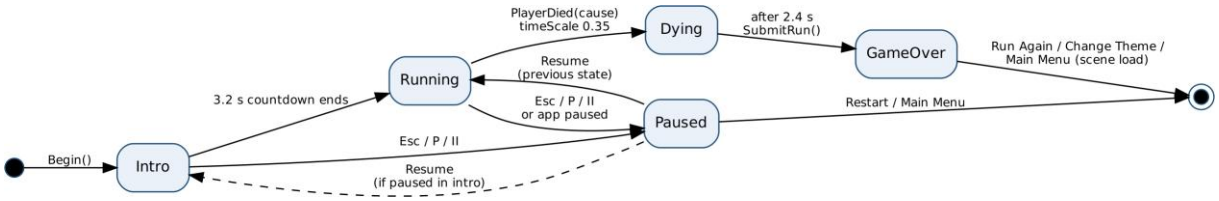


Figure 7.3 *State diagram of a run (GameManager.State).*

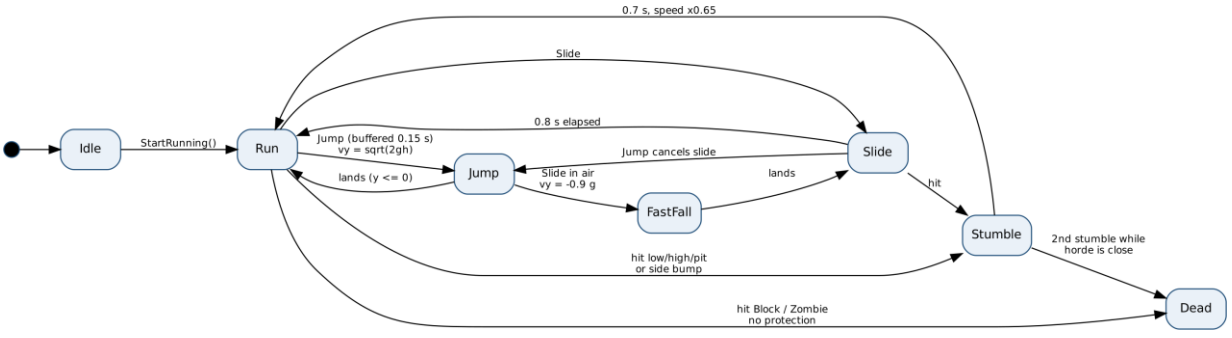


Figure 7.4 *State diagram of the survivor (PlayerController and poses).*

7.4 Gameplay Flowchart

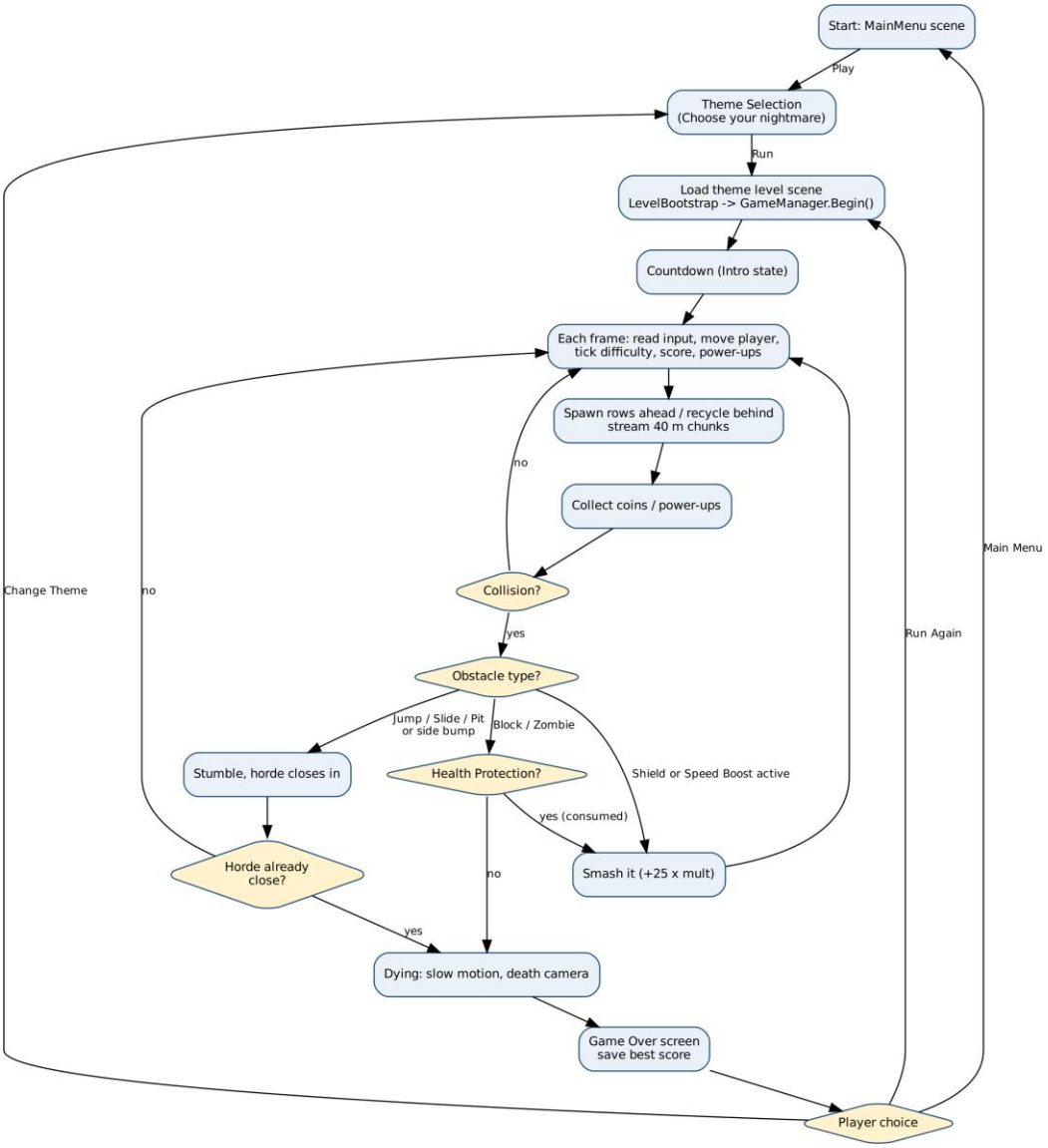


Figure 7.5 *Gameplay flowchart, from the menu through a run to the game-over choices.*

7.5 Communication between GameObjects and C# Scripts

Scripts communicate in three ways, chosen to keep coupling low:

- **C# events** for loose notifications: `InputRouter.OnCommand`, `ScoreManager.OnCheckpoint`, `PowerUpManager.OnEnded` and `GameSettings.OnChanged`. The HUD and audio react to them without the game logic knowing about the UI.
- **Direct references from the owner**: GameManager creates each subsystem and keeps a reference, and subsystems call back through it (for example `gm.CollectCoin()` and `gm.PlayerDied(cause)`).
- **Static services** for global, stateless needs: `AudioManager.PlaySfx`, `SceneFlow.LoadLevel` and `GameSettings` (PlayerPrefs).

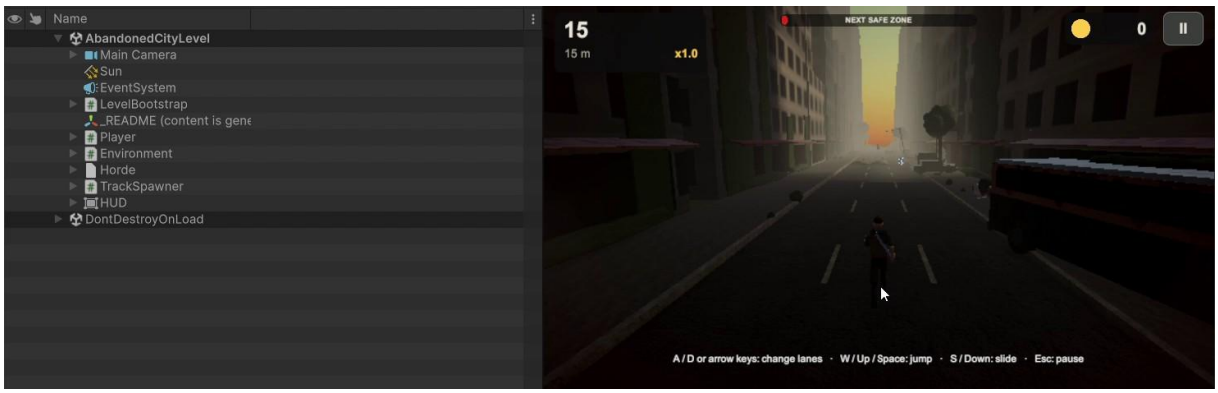


Figure 7.6 *Unity Hierarchy during play (recorded). Player, Environment, Horde, TrackSpawner and HUD were all created at run time by LevelBootstrap and GameManager.*

CHAPTER 8

Game Development Using Unity and C#

8.1 Installing Unity and Creating the Project

- Install Unity Hub and the Unity 6.6 Editor (6000.6.0f1), adding Android Build Support (with OpenJDK, SDK and NDK) and Windows Build Support.
- In Unity Hub choose **New project** and select the **Universal 3D** template, which sets up URP.
- Copy the `Assets/ZombieRunner` folder into the new project and wait for the scripts to compile.
- Run **Tools > Zombie Runner > Setup Wizard > RUN FULL SETUP**, then open the MainMenu scene and press Play.

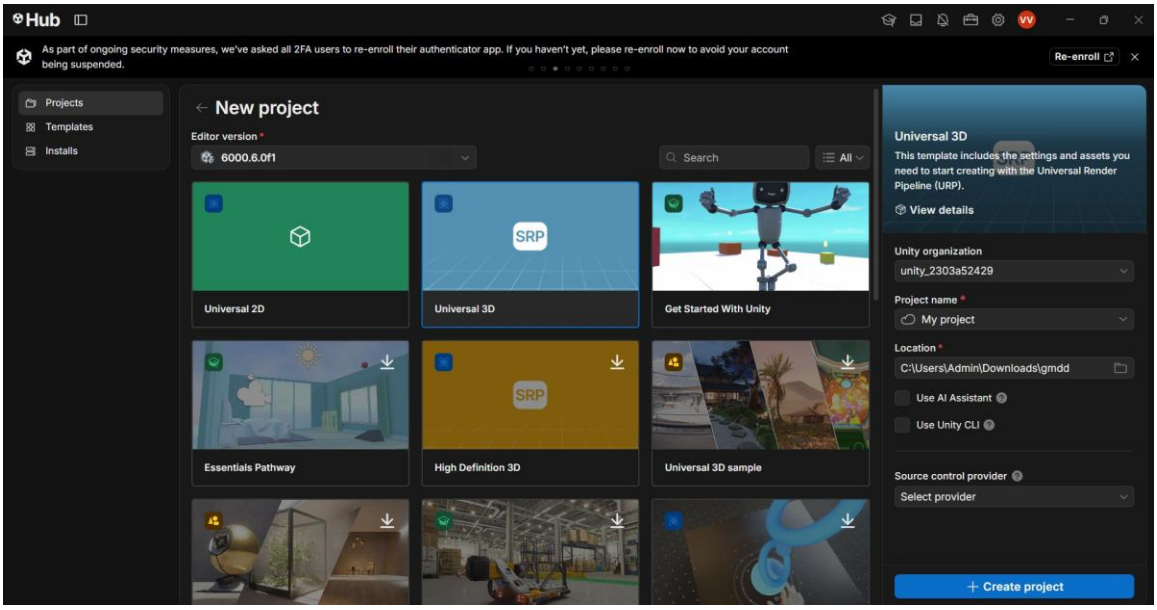


Figure 8.1 *Creating the project from the Universal 3D (URP) template in Unity Hub (screenshot from the team's machine).*

8.2 Project Configuration with the Setup Wizard

The Editor wizard (`ZombieRunnerSetup.cs` and `ZRSetup.cs`, about 1,250 lines) automates configuration that is normally done by hand. It checks that URP and an input backend are active, and creates a URP asset if none is. It creates the folder structure, sorts imported files, configures importers, creates URP materials, the asset registry, Animator Controllers and prefabs, builds the six scenes and adds them to Build Settings in order. It also sets the product name, landscape orientation, Android IL2CPP and ARM64. It ends with a colour-coded validation log.

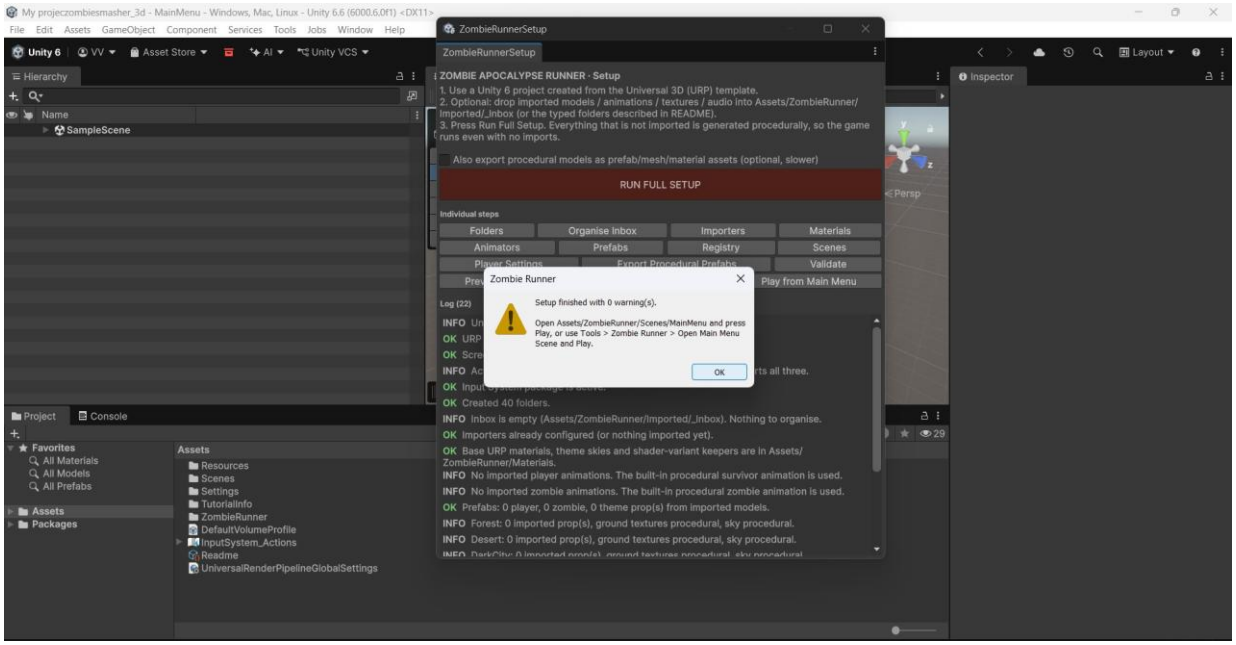


Figure 8.2 *The setup wizard after RUN FULL SETUP in Unity 6.6: "Setup finished with 0 warning(s)" (screenshot from the team's machine).*

8.3 Scene Development

There are six scenes. The two menu scenes contain a `MenuBootstrap` that builds a live 3D diorama of the theme behind the UI. Each level scene contains a `LevelBootstrap` with its theme set. Building content from code instead of saving it in scenes means that the scenes cannot lose references, and the same code can serve all four themes.

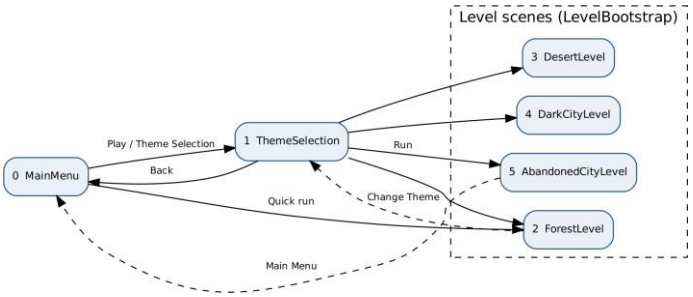


Figure 8.3 *Scene flow and Build Settings order.*

8.4 GameObjects, Components and Prefabs

A Unity scene is a tree of GameObjects, and behaviour is added by attaching components. The runner uses built-in components (Transform, MeshFilter, MeshRenderer, BoxCollider, Light, ParticleSystem, AudioSource, Camera, Canvas) and custom MonoBehaviours (PlayerController, ZombieAgent, Obstacle, Coin, PowerUpPickup, Spinner, FlickerLight). Procedural objects are built by a fluent `PropBuilder`, which collects shapes and materials and then combines them into one mesh per material:

```
var b = new PropBuilder();
b.Cyl(Vector3.zero, 0.32f, 0.07f, Palette.Gold, new Vector3(90, 0, 0), 20);
var vis = b.Build("CoinMesh", root.transform, false);
vis.AddComponent<Spinner>().degreesPerSecond = new Vector3(0, 220, 0);
root.AddComponent<Coin>();
```

Instead of prefab assets, the game uses **factory functions** registered with the ObjectPool. They act as code prefabs: a key such as "Coin" or "Z_Runner_3" maps to a function that builds a new instance whenever the pool runs empty. If you choose to import external models, the wizard can also export these procedural models as real prefab assets.

8.5 Scripting Structure

Table 8.1 *Source code organisation (34 C# files, about 8,750 lines)*

Folder	Responsibility	Key classes
Core	Themes, settings, scene flow, pooling, random numbers, registry	ThemeProfile, GameSettings, SceneFlow, ObjectPool, Rng
Gameplay	Player, input, spawner, zombies, power-ups, run manager	GameManager, PlayerController, InputRouter, TrackSpawner, ZombieAgent, ChaseSystem
Environment	Chunks, props, particle effects	EnvironmentGenerator, PropFactory, FX
Characters	Procedural humanoids, imported-model wrapper	ProceduralHumanoid, CharacterVisual
Rendering	Textures, meshes, materials, lighting, quality	ProceduralTextures, MeshGen, MaterialLibrary, ThemeLighting, QualityManager
Audio	Synthesis and playback	ProceduralAudio, AudioManager
UI	UI factory, HUD, menus	UIFactory, HUDController, MainMenuController, ThemeSelectController
Editor	Setup wizard	ZombieRunnerSetup, ZRSetup

8.6 Input Handling

`InputRouter` turns raw input into five commands: Left, Right, Jump, Slide and Pause. It uses the new Input System when it is active and falls back to the legacy Input Manager otherwise, through compile-time symbols (`ENABLE_INPUT_SYSTEM`). A swipe counts when the finger or mouse moves at least 0.25 inch (minimum 40 px) within 0.6 s, and the dominant axis selects the command. Touches that start in the top 14% of the screen are ignored, so pressing the pause button does not also trigger a move. Because the rest of the game only sees commands, the same gameplay code works for keyboard, gamepad, touch and mouse.

8.7 Gameplay Logic

GameManager.Update runs the state machine. In *Running* it advances difficulty, power-up timers and score, updates the visual effects on the player and checks the floating origin. PlayerController.Update handles buffered input, gravity, lane movement and forward movement, then checks for collisions. TrackSpawner.Update spawns rows until the track is filled 170 m ahead, recycles objects behind the player and handles pickups. Keeping each responsibility in one Update method makes the order of operations easy to follow and debug.

CHAPTER 9

Physics, Animation and Collision Detection

9.1 Gravity and Movement

An endless runner needs exact, repeatable movement more than realistic simulation. Running the player through Unity's Rigidbody solver would add bounces, friction and frame-rate-dependent results that make tuning harder. The survivor is therefore moved **kinematically** in code. Vertical motion integrates a custom gravity of 38 m/s^2 each frame, horizontal motion moves toward the lane centre at a fixed rate, and forward motion uses the speed from DifficultyManager. Unity's physics gravity (`Physics.gravity`) is still used for the arc of obstacles that are knocked away, where natural motion looks better.

9.2 Rigidbody and Collider Usage

Obstacles, zombies and pickups have **BoxCollider** components marked as triggers, but no Rigidbody. Their size is set by the obstacle type rather than by the art (Figure 9.1). The player has no collider at all. Instead, each frame it runs a **swept overlap query**: a box covering the player's height (1.7 m standing, 0.75 m sliding) and stretched along the distance moved in that frame. Stretching the box means fast movement cannot skip through a thin obstacle (tunnelling), whatever the frame rate.

```
float height = Sliding ? 0.75f : 1.7f;
var center = new Vector3(x, y + height * 0.5f + 0.05f, (z0 + z1) * 0.5f);
var half   = new Vector3(0.28f, height * 0.5f, 0.25f + dz * 0.5f);
Physics.SyncTransforms();
int n = Physics.OverlapBoxNonAlloc(center, half, hits, Quaternion.identity,
                                   ~0, QueryTriggerInteraction.Collide);
```

`Physics.SyncTransforms()` is needed because pooled obstacles and zombies are moved by setting their Transform. Without it, the physics scene might still hold their old positions when the query runs. `OverlapBoxNonAlloc` writes into a preallocated array of 16 results, so collision detection allocates no memory.

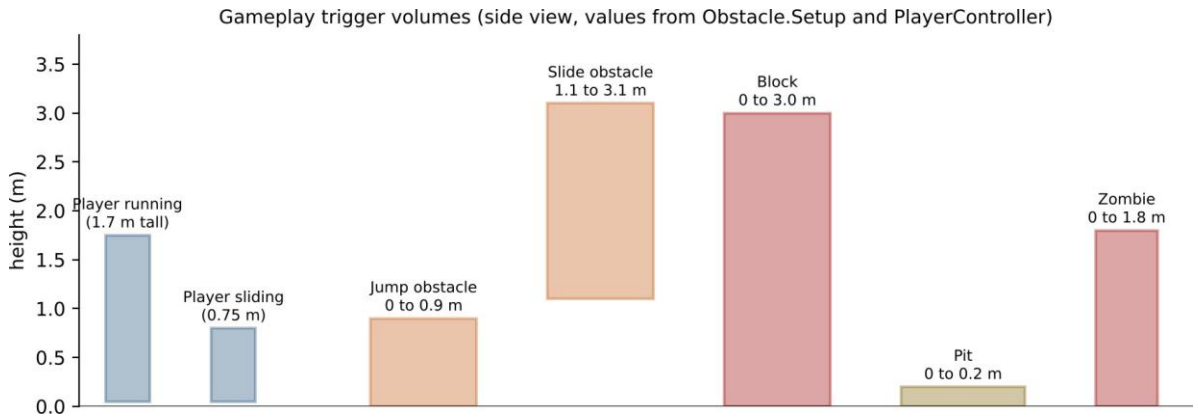


Figure 9.1 *Trigger volumes used for gameplay, compared with the player's standing and sliding boxes.*

9.3 Physics Materials

Because the player is moved kinematically and obstacles are triggers, no surfaces in the game rely on friction or bounce, so no Physics Materials are used. The "material" of a surface affects gameplay only through sound: each theme's road style (mud, desert road, wet asphalt, cracked asphalt) selects a set of four footstep sounds, and footstep timing scales with run speed.

9.4 Animator Controllers

With procedural characters, poses are applied directly by `ProceduralHumanoid`. With imported rigged characters, `CharacterVisual` drives an Animator Controller created by the wizard. It uses the float parameter *Speed*, the bools *Grounded* and *Sliding*, and the triggers *Jump, Slide, Stumble, Die, Attack* and *Rise*. Gameplay code only calls `visual.SetPose(Pose.Jump)`, so it does not need to know which kind of character is in use.

9.5 Trigger-Based Events

- **Obstacle contact**: the overlap query finds an Obstacle, and HandleHit applies the rules in Table 4.1.
- **Zombie activation**: an Emerger stays hidden until the player is within 42 m, then rises. Its collider is enabled only once it has partly risen (after 0.45 s), so it cannot kill the player before it is visible.
- **Pickups**: distance checks against the player's centre (1.15 m for coins, 1.4 m for power-ups) trigger collection, a particle burst and a sound.
- **Smash**: when the player is invulnerable, contact disables the obstacle's collider and flings its visual with random spin under gravity, shrinking it away over 0.4 s.

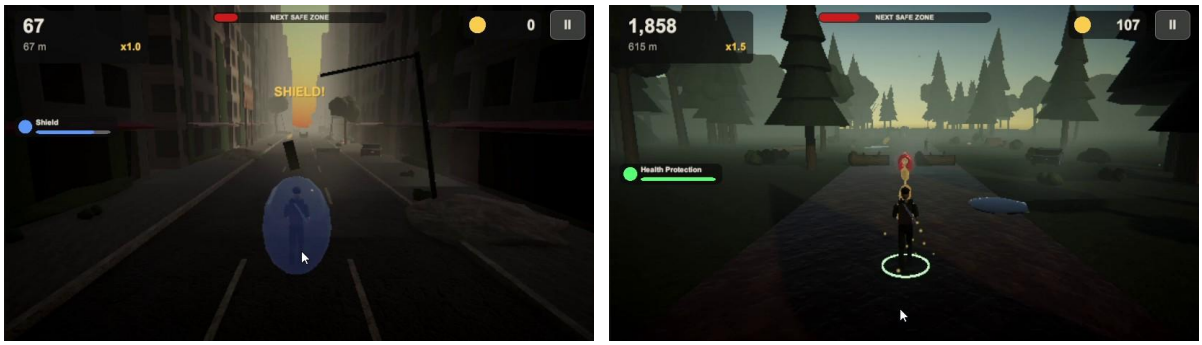


Figure 9.2 *Power-up feedback in the recordings. Left: Shield bubble around the survivor in the Abandoned City. Right: Coin Magnet ring (red) and Health Protection ring (green) in the Forest.*

CHAPTER 10

User Interface and User Experience Design

10.1 Design Principles

- **Readable at a glance:** large bold numbers, high contrast and dark translucent panels over the 3D scene.
- **Few choices per screen:** one red primary button per screen (PLAY, RUN, RESUME, RUN AGAIN) and grey secondary buttons.
- **Feedback for every event:** short toast messages, sounds and camera shake confirm pickups, endings of power-ups and safe zones.
- **Built in code:** `UIFactory` creates all Canvas, Text and Button objects with a shared style, so every screen looks consistent and nothing has to be wired by hand.

10.2 Main Menu

The main menu shows the game title over a live 3D diorama, with buttons for **PLAY**, a **Quick Run** into the last theme, **THEME SELECTION**, **CHARACTER**, **SETTINGS** and **QUIT** (desktop only). It also shows the best score, best distance, coins banked and number of runs. The Character panel ("CHOOSE YOUR SURVIVOR") previews the three survivors. The Settings panel holds music and effects volume, mute, a quality tier (Low, Medium, High or Auto), an FPS counter switch and a two-tap **RESET PROGRESS** button that guards against accidental resets.

10.3 Theme Selection

"CHOOSE YOUR NIGHTMARE" shows four cards, one per theme. Each card has a live preview with zombies, the theme name and tagline, the best score for that theme and a RUN button. The recorded session shows these best scores updating after each run.



Figure 10.1 *Theme selection screen (recorded) showing the best score kept separately for each theme: 348, 401, 381 and 448.*

10.4 Gameplay HUD, Score Display and Health Indicators

The HUD shows, from left to right: score (large), distance and multiplier; the "NEXT SAFE ZONE" progress bar; the coin counter; and the pause button "II". Active power-ups appear on the left as coloured slots with a draining timer bar. A centre label shows the countdown and "RUN!", and the toast line announces events. Instead of a health bar, the **danger overlay** tints the screen edges red as the horde approaches. A control hint is shown at the start of each run, and an optional FPS counter appears in the corner.

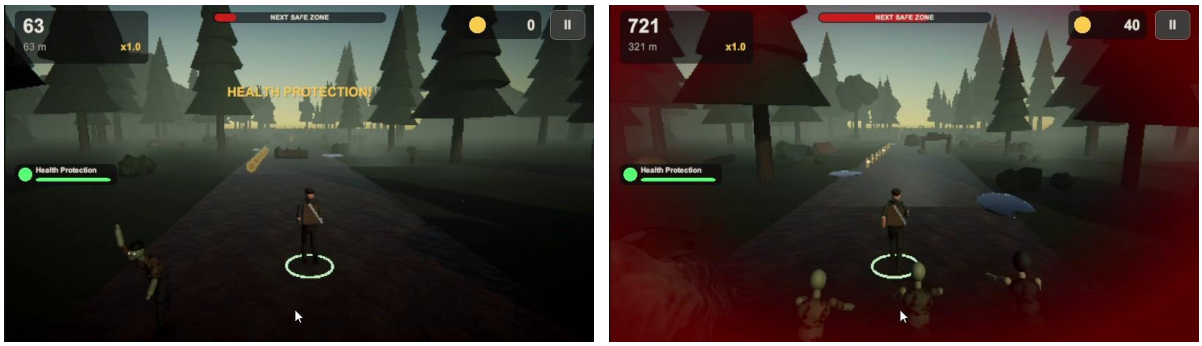


Figure 10.2 *HUD in the Forest (recorded). Left: "HEALTH PROTECTION!" toast and power-up slot. Right: red danger overlay with zombies closing in after a stumble.*

10.5 Instructions

The game teaches through play rather than a tutorial screen. The control hint matches the platform: keys on PC ("A / D or arrow keys: change lanes · W / Up / Space: jump · S / Down: slide · Esc: pause") and swipe directions on mobile. Coin lines then show where to go and whether to jump or slide.

10.6 Pause and Restart Controls

Esc, P, the gamepad Start button or the on-screen "II" opens the **PAUSED** panel with RESUME, RESTART RUN and MAIN MENU. Pausing sets the time scale to zero and pauses all audio. The game also pauses automatically when a mobile app goes to the background, and resuming returns to the exact previous state, including the countdown if the game was paused during it.

10.7 Game-Over Screen

After a death the game plays 2.4 s of slow motion (time scale 0.35) while the camera orbits 140 degrees around the survivor and the horde attacks. Then "YOU DIDN'T MAKE IT" appears with the cause of death, "NEW BEST SCORE!" when relevant, the final score, distance, coins and best score, and three buttons: RUN AGAIN, CHANGE THEME and MAIN MENU.



Figure 10.3 *End of the Forest run (recorded): the horde catches the survivor (left), then the game-over screen with 2,654 points, 845 m and 137 coins (right).*

CHAPTER 11

Audio and Visual Effects Integration

11.1 Procedural Audio

All audio is synthesised by `ProceduralAudio` when the game first loads, as mono clips at 22,050 Hz. Sound effects are built from oscillators, filtered noise and envelopes. Growls use low-frequency oscillators with rough modulation. The music loops are generated from a seeded style per theme. Any sound can be replaced by putting an audio file named after its key (for example `coin.wav` or `music_forest.ogg`) in the Imported/Audio folder.

Table 11.1 *Audio content by category (sound keys)*

Category	Keys
Background music	music_menu, music_forest, music_desert, music_darkcity, music_abandoned
Environmental ambience	amb_forest, amb_desert, amb_city, amb_ruins; chase_loop
Player and object SFX	jump, land, slide, whoosh, coin, bump, crash, smash, grunt, footstep_[surface]_0..3
Zombies	zombie_growl, brute_roar, zombie_attack, zombie_hit
Power-ups and UI	powerup_magnet, powerup_speedboost, powerup_shield, powerup_doublecoins, powerup_healthprotection, powerdown, checkpoint, countdown, go, gameover, ui_click

11.2 Audio Playback Architecture

`AudioManager` is a persistent object that survives scene loads. It has two music sources for **cross-fading** between tracks, one ambience source, one chase-layer source, one UI source, a pool of 10 two-dimensional SFX sources and a pool of 8 three-dimensional sources for positioned sounds such as zombie growls. Small random pitch variations (for example 0.95 to 1.05 on jumps) stop repeated sounds from feeling mechanical.

11.3 Interactive Gameplay Audio

The **chase layer** is the main piece of interactive audio. Its volume follows the horde's Danger value, so the music grows more intense as the zombies close in after a stumble and fades as they fall back. Footstep timing is tied to run speed, the countdown plays a tick each second, and on death the music stops and is replaced by the attack and game-over sounds.

11.4 Particle Effects

Table 11.2 *Particle systems (FX.cs) and where they are used*

Effect	Use
Rain	Spooky Dark City weather, following the player
Sandstorm	Desert weather; also a swirl over desert sand pits

Effect	Use
Ground fog, light shafts, falling leaves	Forest atmosphere
Fire, smoke, smoke columns, ash	Abandoned City fires, burning cars, ruins
Bursts	Coin pickup (gold), power-up pickup (coloured), smash debris, slide dust, dirt when an Emerger rises
Motes and speed lines	Power-up sparkles; stretched speed lines in front of the camera during Speed Boost

11.5 Explosions, Impacts and Lighting Effects

The game has no explosions. Impacts are shown by smashing obstacles away, with a dust burst, a flung and spinning model and camera shake. Lighting effects include a flickering red beacon on police roadblocks, flickering neon signs, fire lights, emissive glowing power-ups with transparent halos, and per-theme bloom that makes neon and fire glow. Point lights and their flicker are only enabled when the quality tier allows extra lights.

11.6 Synchronisation with Gameplay Events

Table 11.3 *Feedback triggered by each gameplay event*

Event	Audio	Visual	Camera / UI
Coin collected	coin (higher pitch when doubled)	Gold burst	Counter updates
Power-up collected	Power-up specific sound	Coloured burst; shield, ring or speed lines	Toast, slot with timer, small shake
Stumble	bump, grunt, zombie_attack	Stumble pose	Shake, red danger overlay
Smash	smash / zombie_hit	Debris burst, flung model	Shake; +25 x multiplier
Safe zone	checkpoint	-	"SAFE ZONE n REACHED" toast, multiplier rises
Death	zombie_attack, gameover; music stops	Horde attack pose	Slow motion, orbiting death camera, game-over screen

All feedback for an event is fired from the same function (for example `GameManager.CollectPowerUp`), so sound, effects and UI always happen in the same frame as the rule that caused them.

CHAPTER 12

Gameplay Implementation and Integration

12.1 Integrating the Subsystems

Integration happens in `GameManager.Begin()`, which builds a complete run in a fixed order: (1) quality settings and the theme profile; (2) lighting, sky, fog and the post-processing volume; (3) the player and its effects; (4) the camera rig; (5) the environment chunks and weather; (6) the horde and the track spawner; (7) input and the HUD; (8) event subscriptions; (9) music and ambience. Because one function builds everything, every level scene is identical except for its theme value.

12.2 Gameplay Sequence

- Theme selected, then the level scene loads and LevelBootstrap calls GameManager.Begin().
- Intro: a 3.2 s countdown ("4, 3, 2, 1, RUN!"). The horde stands close behind.
- Running: the horde drops back to 13 m, and rows of obstacles, zombies, coins and power-ups stream in.
- Events (pickups, stumbles, smashes, safe zones) update the run logic and fire feedback.
- Death: slow motion and the death camera, then the run is saved and the game-over screen appears.

12.3 Row Generation Algorithm

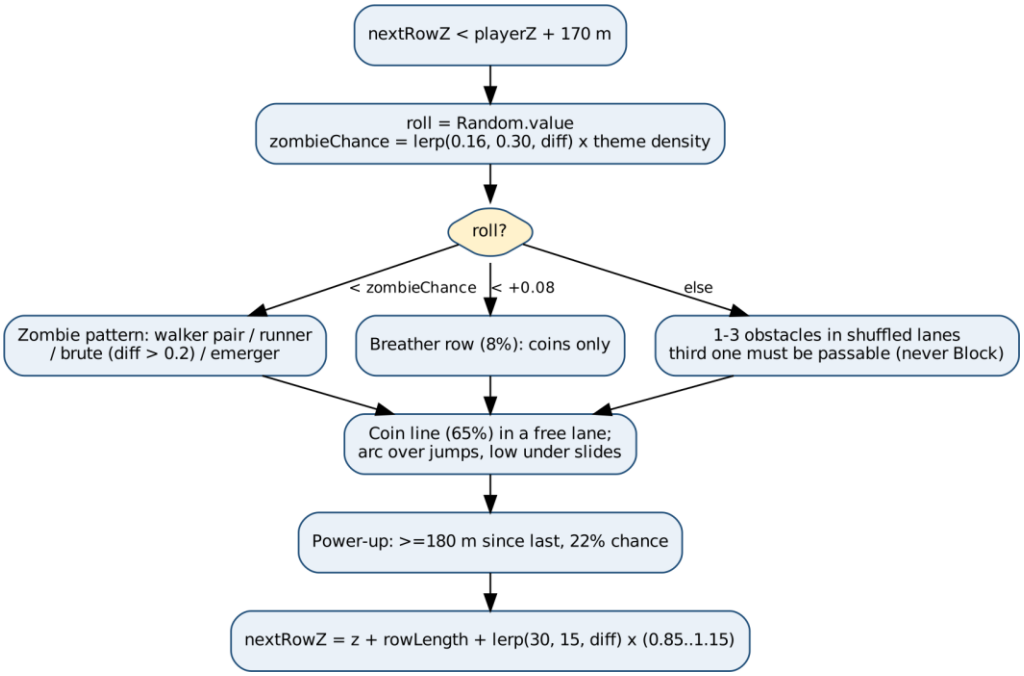


Figure 12.1 *Row generation in TrackSpawner.SpawnRow.*

12.4 Scene Transitions

`SceneFlow` loads scenes by name. Before each load it resets the time scale and audio pause, and it reports a clear error if the scene is missing from Build Settings. RUN AGAIN reloads the current scene, CHANGE THEME returns to theme selection, and MAIN MENU returns to the first scene. Static caches are cleared when Play mode starts (`ZRStaticReset`), so the game also works when Unity's "Enter Play Mode Options" skips the domain reload.

12.5 Level Completion and Player Progression

There are no fixed levels to complete. Progress is measured in **safe zones** within a run, and in **saved records** across runs. `GameSettings.SubmitRun` stores the best score per theme, the overall best score and distance, total coins banked and the number of runs in PlayerPrefs. These values appear on the main menu and the theme cards, giving long-term goals.

12.6 Overall Game Execution and Long Runs

Two techniques keep the game stable however long a run lasts. **Floating origin:** when the player passes z = 2000 m, the player, camera, environment and all spawned objects are shifted back by 2000 m in one frame. 2000 is a multiple of the 40 m chunk length, so the shift cannot be seen. **Pooling and recycling:** after warm-up, rows and chunks reuse existing objects, so a long run does not keep creating new ones.

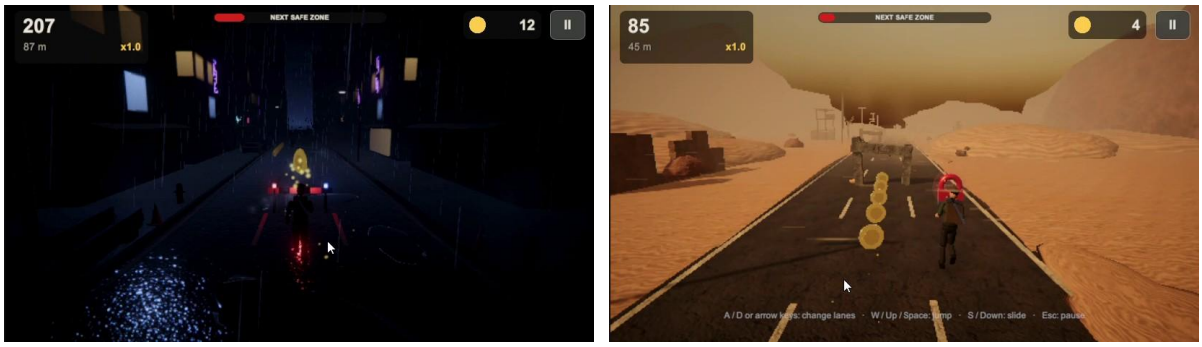


Figure 12.2 *Integrated gameplay (recorded). Left: Dark City with a police roadblock, rain and coin line. Right: Desert with Coin Magnet active and coins being pulled in.*

CHAPTER 13

Game Testing, Debugging and Optimization

Testing status. The game was run and played in the Unity 6.6 Editor on Windows (DX11), and the session was screen-recorded. The results below marked **Observed** are visible in those recordings. Items marked **Not yet verified** were not captured, and items marked **Not measured** need tools such as the Unity Profiler. No device builds have been tested yet. No figures in this report are estimated or invented.

13.1 Functional Test Cases

Table 13.1 *Functional test cases*

ID	Test	Expected result	Status
F1	Select each theme and press RUN	Correct themed level loads with countdown	Observed (all 4 themes)
F2	Countdown	4-3-2-1, then "RUN!" and the run starts	Observed
F3	Coin collection	Counter increases; score +10 x multiplier	Observed
F4	Coin Magnet	Coins pulled to the player for 10 s; "Coin Magnet ended" toast	Observed (Desert, Forest)
F5	Shield	Bubble shown for 8 s; "Shield ended" toast	Observed (Abandoned City)
F6	Health Protection	Green ring and slot until used	Observed (Forest)
F7	Speed Boost, Double Coins	Speed x1.45 / coins worth 2	Not yet verified
F8	Safe zone at 500 m	Toast, multiplier x1.0 to x1.5	Observed (Forest, about 507 m)
F9	Grabbed by a zombie	Death, cause shown	Observed (Dark City, Desert)
F10	Second stumble near horde	"The horde caught you"	Observed (Abandoned City, Forest)
F11	Game-over statistics and new best	Correct score, distance, coins; "NEW BEST SCORE!"	Observed
F12	Best score per theme	Theme cards show separate bests	Observed (348 / 401 / 381 / 448)
F13	Pause, resume, restart	Time stops; buttons work	Not yet verified

13.2 Collision, Input and UI Testing

Table 13.2 *Collision, input and UI tests*

Area	Test	Status
Collision	Fatal contact with zombies ends the run with the right cause	Observed

Area	Test	Status
Collision	Stumble on low/high obstacle brings the horde close (red overlay)	Observed (danger overlay visible)
Collision	Jump clears low obstacles; slide clears overhead ones	Not yet verified frame by frame
Collision	Side-bump returns the player to the previous lane	Not yet verified
Input	Keyboard lane changes (A/D, arrows)	Observed (lane changes in the recordings)
Input	Mouse clicks on UI buttons (RUN, RUN AGAIN, CHANGE THEME)	Observed
Input	Touch swipes and gamepad	Not yet verified (needs device)
UI	HUD values update live (score, distance, multiplier, coins)	Observed
UI	Main menu layout	Defect found in an early build (see 13.4)

13.3 Gameplay Balancing

The recorded runs give a first check of the balance. The three shorter runs ended between 151 m and 288 m, and all three were ended by zombies or the horde rather than by blocks. The Forest run, with Health Protection collected early, reached 845 m. This suggests that zombies are the main threat in the early game, which matches the design intent. Four runs are too few to draw firm conclusions. Further balancing should log the cause of death and distance for 20 or more runs per theme, and then adjust `zombieChance`, row spacing and `ChaseSystem.closeDuration`.

13.4 Bug Identification and Debugging

Table 13.3 *Issues found during development*

Issue	Cause	Resolution
"Packages with Errors" when opening the unzipped folder directly	No Packages/manifest.json, so Unity created one without URP and uGUI	Added a manifest with URP, uGUI and Input System. The wizard creates a URP asset if none is active.
Input backend prompt on first open	The project uses the new Input System package	Both answers work, because InputRouter supports both backends
Main menu text overlapping buttons and washed-out background in an early build (Figure 13.1)	Layout and rendering problem in the early menu build	The later recordings show the theme selection, HUD and game-over screens rendering correctly. A new main-menu screenshot should be taken to confirm the fix.
UI not receiving clicks when duplicate EventSystems exist	Duplicates steal focus; a code-created input module can lack an actions asset	LevelBootstrap.EnsureEventSystem keeps one and assigns default actions

Issue	Cause	Resolution
Overlap query missing pooled objects	Objects moved by Transform were not yet synced to physics	Physics.SyncTransforms() before the query
Stale static state between Play sessions	Domain reload disabled in Enter Play Mode Options	ZRStaticReset clears caches when Play mode starts

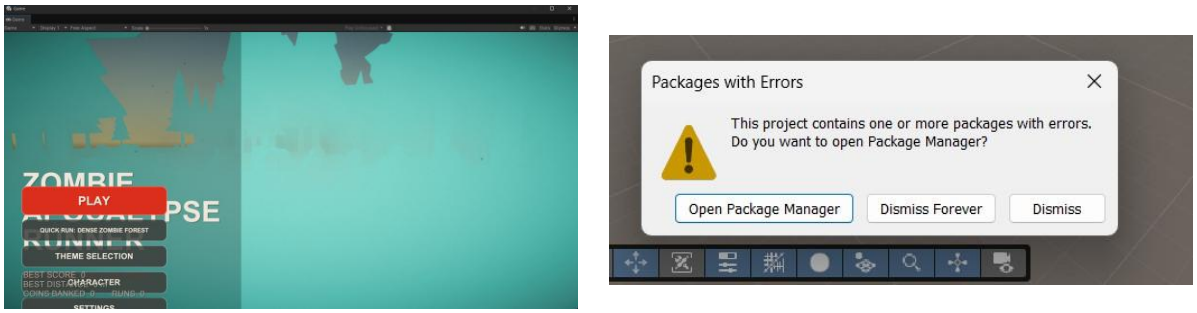


Figure 13.1 *Left: early main-menu build with overlapping text and a washed-out background. Right: the "Packages with Errors" dialog (screenshots from the team's machine).*

13.5 Performance Optimisation

- **Object pooling:** obstacles, coins (40 pre-warmed), power-ups, zombies and effects are reused, so no objects are created during a run after warm-up.
- **Chunk streaming:** only about 6 chunks (240 m) exist at once, and chunks behind the player are recycled.
- **Static mesh merging:** each chunk's scenery becomes a few meshes, one per material, which reduces draw calls.
- **Culling:** 450 m camera far plane, frustum culling, fog that hides the far end and a shadow distance set per quality tier.
- **Allocation-free queries:** OverlapBoxNonAlloc with a preallocated buffer.
- **Quality tiers** change render scale, shadow distance (35 / 60 / 90 m), cascades, MSAA, extra lights, particle density (40% / 70% / 100%), decor density, horde size and texture resolution.

13.6 Frame-Rate Monitoring and Memory Usage

The game includes its own monitoring. The **FPS counter** (Settings > Show FPS) displays a value averaged over half a second. In **Auto** quality mode, QualityManager measures the frame rate and lowers the render scale and preset when it stays below 50 FPS, or raises it again above 58 FPS, aiming for 60 FPS.

Not measured. Frame rate and memory use have not been recorded for this report, so no performance graphs are included. To add them: enable Show FPS and capture each theme on Low, Medium and High; use Window > Analysis > Profiler (CPU, Rendering, Memory modules) during a 3-minute run; and repeat on an Android phone with a development build.

CHAPTER 14

Results and Performance Evaluation

14.1 Final Implementation

The final game runs in Unity 6.6 in all four themes. The complete flow worked in the recorded session: theme selection, countdown, gameplay with coins, power-ups, zombies and the horde, death, the game-over screen, and back to theme selection. The screenshots in this chapter are frames from that recording.

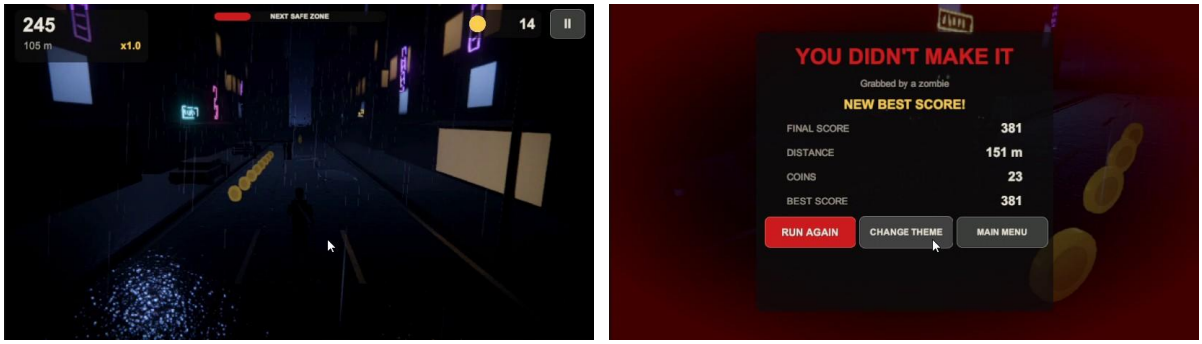


Figure 14.1 *Spooky Dark City: night, rain and neon (left); game over after being grabbed by a zombie at 151 m (right).*

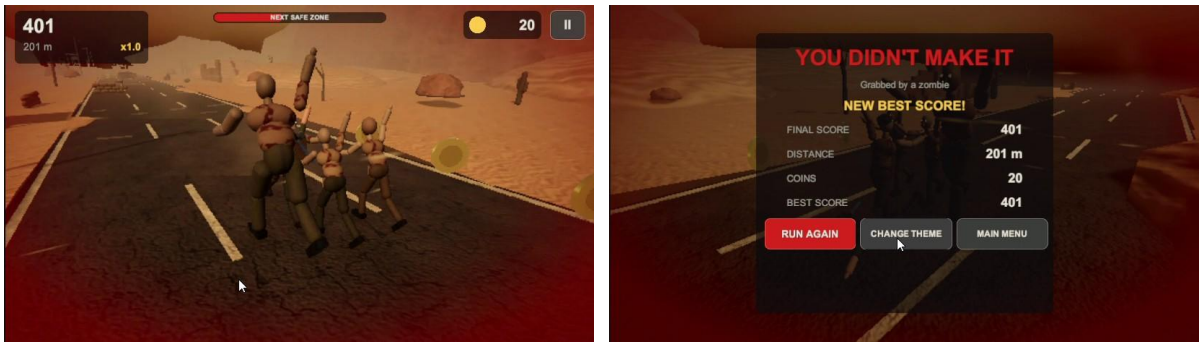


Figure 14.2 *Post-Apocalyptic Desert: zombies reach the survivor (left); game over at 201 m with 401 points (right).*

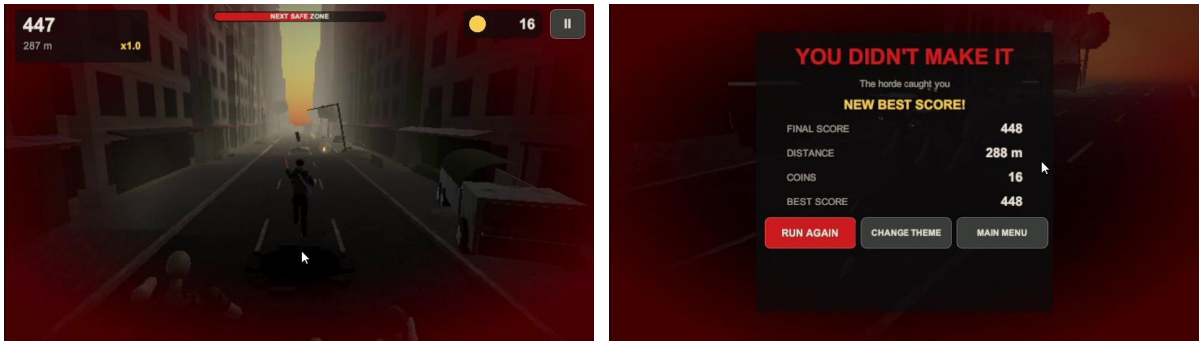


Figure 14.3 *Abandoned Apocalyptic City: the horde closes in (left); "The horde caught you" at 288 m (right).*

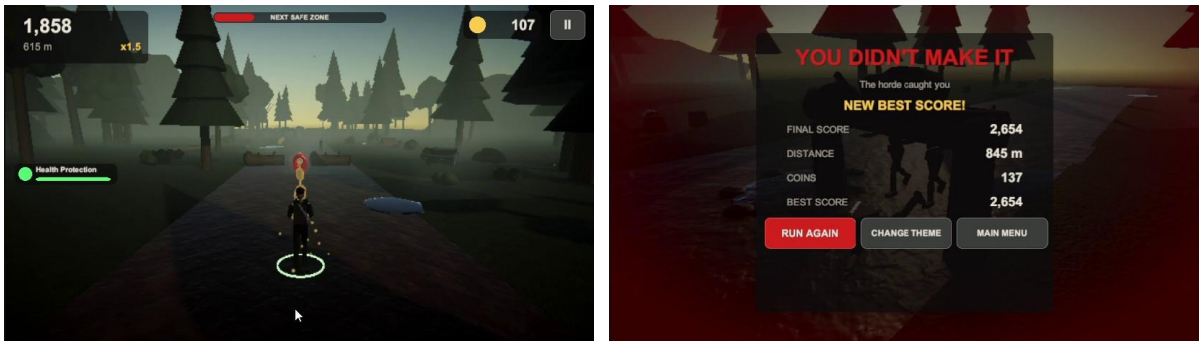


Figure 14.4 *Dense Zombie Forest: Coin Magnet and Health Protection active at x1.5 (left); final result of 2,654 points over 845 m (right).*

14.2 Observed Results

Table 14.1 *Observed results of the recorded runs (read from the game-over screens)*

Theme	Score	Distance	Coins	Cause of death	Score check (Section 4.5)
Spooky Dark City	381	151 m	23	Grabbed by a zombie	151 + 23x10 = 381 (exact)
Post-Apocalyptic Desert	401	201 m	20	Grabbed by a zombie	201 + 20x10 = 401 (exact)
Abandoned Apocalyptic City	448	288 m	16	The horde caught you	288 + 16x10 = 448 (exact)
Dense Zombie Forest	2,654	845 m	137	The horde caught you	Uses x1.5 after 500 m; consistent with the formula*

*The Forest score cannot be checked exactly from the final screen, because it depends on how many coins were collected before and after the 500 m safe zone. The HUD in the recording shows 84 coins at 512 m. Taking that as the split gives 500 + 345 x 1.5 + 84 x 10 + 53 x 10 x 1.5 = 2,652.5, within three points of the 2,654 shown. The small gap comes from rounding and the coins collected between 500 m and 512 m.

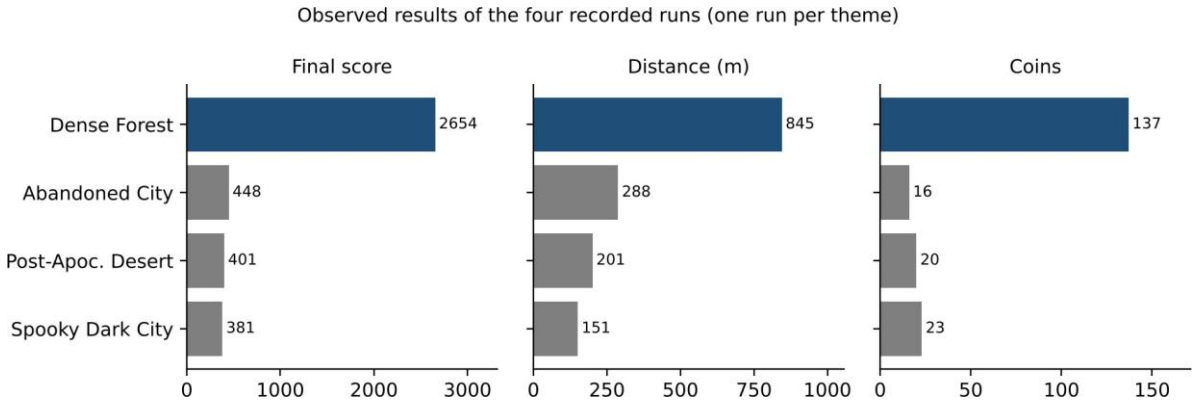


Figure 14.5 *Observed score, distance and coins of the four recorded runs. This is one run per theme, so it shows that the game works but does not compare theme difficulty.*

Speed model check. In the recording, the Forest run lasts about 60 seconds from "RUN!" to the death. Integrating the speed curve over 60 s predicts about 853 m, close to the 845 m observed (Figure 4.3). The difference is explained by the stumbles, which slow the player to 65% speed

for 0.7 s each. This suggests that the difficulty ramp behaves in play as designed.

14.3 Performance Evaluation

Performance graphs are not included, because frame rate and memory use were not measured. The recordings show smooth play in the Editor, but screen capture is not a measurement, so this report makes no frame-rate claim. Section 13.6 describes how to collect these numbers using the built-in FPS counter and the Unity Profiler.

14.4 Player Feedback

No structured player feedback has been collected yet. A short play-test is recommended: 5 to 10 classmates each play two runs in two themes and then answer a questionnaire on control responsiveness, fairness of deaths, readability of obstacles, and preferred theme (1 to 5 scale). The results would fit in this section as a table and a bar chart.

14.5 Comparison with the Objectives

Table 14.2 *Objectives versus outcome*

Objective	Outcome	Evidence
O1 Core runner movement	Achieved	Lane changes, stumbles and deaths in the recordings; jump and slide implemented (not separately captured)
O2 Four themed worlds	Achieved	All four themes recorded (Figures 14.1-14.4)
O3 Zombies, horde, coins, power-ups	Achieved	Zombie deaths, horde catches, coins, Magnet, Shield and Health Protection recorded
O4 Fair difficulty curve	Achieved in design; balancing ongoing	Speed model matches the Forest run; more runs needed (13.3)
O5 Complete UI flow with saved bests	Achieved	Theme cards show per-theme bests; game-over flow recorded
O6 Audio and visual feedback	Implemented	Particles, overlays and toasts visible; audio not assessable from the report
O7 Efficiency features	Implemented; not measured	Pooling, streaming, merging and tiers in code; profiling pending
O8 Automated setup	Achieved	Wizard reports "Setup finished with 0 warning(s)" (Figure 8.2)

CHAPTER 15

Project Conclusion and Future Enhancements

15.1 Summary of Accomplishments

Zombie Smasher 3D is a complete, playable 3D endless runner in Unity 6.6 URP. It has four themed worlds, three survivors, five zombie behaviours, a pursuing horde, five power-ups, a scaling difficulty curve, a full menu and HUD flow with saved records, and procedurally generated models, textures, music and sound. A custom Editor wizard sets the whole project up automatically. The recorded play session shows every major system working, and the scores on the game-over screens match the scoring formula exactly.

15.2 Implemented Mechanics

- Three-lane movement with lane-change bounce, buffered jump and slide, and fast-fall.
- Swept-box collision without a Rigidbody, with type-based rules (jump, slide, block, pit, zombie).
- Stumble-and-chase "health" system with a red danger overlay and a chase music layer.
- Coins, five power-ups, smash bonus, and safe-zone score multiplier.
- Exponential speed ramp driving row density, spacing and zombie mix, with a guaranteed open lane.

15.3 Technical Challenges

- Keeping procedurally generated rows fair at any speed, which needed explicit constraints instead of pure randomness.
- Supporting both the new and legacy input systems, and Unity 6 package versions, in one code base.
- Making collision reliable at up to 36 m/s (Speed Boost at top speed) without tunnelling, which was solved by the swept overlap box.
- Keeping precision and memory stable in an endless world, which was solved by the floating origin and pooling.
- Building a convincing look with no imported art, which relied on per-theme lighting, fog and post-processing.

15.4 Limitations

- The characters and props are stylised and built from primitives, not photo-realistic.
- Performance has not been profiled, and Windows and Android builds have not been tested on devices.
- Several features (Speed Boost, Double Coins, pause and touch input) were not captured in the test recordings.
- No player feedback has been gathered yet, and balancing is based on only four recorded runs.

- Zombies move along lanes rather than using NavMesh pathfinding. This is a deliberate choice for a straight track, but it limits AI variety.

15.5 Learning Outcomes

The project gave practical experience of turning a design into numbers and rules, structuring a Unity game around a state machine and events, procedural generation under constraints, real-time performance techniques, Editor scripting, and an honest testing process that separates observed results from untested ones.

15.6 Future Enhancements

Table 15.1 *Potential improvements*

Enhancement	Approach
Multiplayer	An asynchronous "ghost race" against a friend's recorded run first, then real-time two-player races with Netcode for GameObjects; online leaderboards
Advanced AI	Zombies that predict the player's lane, flank in groups and react to sound; a horde that adapts to the player's skill (dynamic difficulty adjustment)
Additional levels	New themes as extra ThemeProfiles (for example snow or a subway); boss chases; daily challenge seeds
Mobile deployment	Publish to Google Play with tuned Low/Medium tiers, haptic feedback and cloud-saved progress
AR/VR integration	VR mode with head-tilt lane changes and comfort settings; AR "tabletop" mode showing the track on a real surface with AR Foundation
Content and polish	Rigged Mixamo characters through the existing import path, more power-ups, unlockable survivors bought with banked coins

15.7 Conclusion

The project met its main objectives. It produced a complete and stable endless runner that applies the core topics of game mechanics, design and development. Its main design contribution is a set of fairness rules and a stumble-and-chase system that turn the genre's randomness and one-hit failures into tension the player can manage. The next steps are device builds, profiling and structured play-testing, which will provide the performance data and player feedback this report does not yet contain.